



FACULTY OF ENGINEERING AND TECHNOLOGY

Data Structure and Algorithms

Laboratory

(03010503PC02)

3rd SEMESTER

Artificial Intelligence and Data Science

Laboratory Manual

CERTIFICATE

This is certify that

Mr./Ms.....

*With Enrolment Nohas successfully
completed his/her laboratory*

*experiments in the **“Data Structure and Algorithms Laboratory (03010503PC02)”**
from the department of*

Artificial Intelligence and Data Science** During the academic year **2026 – 2027.



Date of Submission

Staff Incharge.....

Head of the Department:.....

TABLE OF CONTENTS

Sr. No	Experiment Title	Page No		Date of Start	Date of Completion	Sign	Marks (out of 20)
		From	To				
1.	Implement Stack and its operations like (creation push pop traverse peek search) using linear data structure						
2.	Implement Infix to Postfix Expression Conversion using Stack						
3.	Implement Postfix evaluation using Stack.						
4.	Implement Towers of Hanoi using Stack.						
5.	Implement queue and its operations like enqueue, dequeue, traverse, search.						
6.	Implement Single Linked lists and its operations (creation insertion deletion traversal search reverse)						
7.	Implement Double Linked lists and its operations (creation insertion deletion traversal search reverse)						
8.	Implement binary search and interpolation search.						
9.	Implement Bubble sort, selection sort, Insertion sort, quick sort, merge sort.						
10.	Implement Binary search Tree and its operations (creation, insertion, deletion)						
11.	Implement Traversals Preorder Inorder Postorder on BST.						
12.	Implement Graphs and represent using adjacency list and adjacency matrix and implement basic operations with traversals (BFS and DFS)						

PRACTICAL-1

Title:

Stack Implementation Using Array with Operations: Push, Pop, Peek, Traverse, and Search

Aim:

To implement a stack using an array in C++ and perform operations like push, pop, peek, traverse, and search.

Purpose:

The purpose of this practical is to understand the Last In First Out (LIFO) concept of a stack and how to implement stack operations using arrays as a linear data structure.

Problem Statement:

Write a C++ program to implement a stack using array and perform the following operations:

Push (insert element)

Pop (remove top element)

Peek (see top element)

Traverse (display all elements)

Search (find a specific element)

SOURCE CODE

```
#include
<iostream>
using namespace
std; #define SIZE
100 class Stack {
int arr[SIZE];
int top; public:
Stack() {
    top = -1;
}
// Push element    void
push(int value) {    if (top ==
SIZE - 1) {        cout <<
"Stack Overflow\n";
return;
}
```

```

        arr[++top] = value;
        cout << value << " pushed to stack\n";
    }
// Pop element    void pop() { DATAFACULTY

        if (top == -1) {
            cout << "Stack Underflow\n";
return;
        }
        cout << arr[top--] << " popped from stack\n";
    }
// Peek element
void peek() {        if
(top == -1) {
        cout << "Stack is empty\n";
        return;
    }
    cout << "Top element is: " << arr[top] << endl;
}

// Traverse stack    void
traverse() {        if (top
== -1) {
        cout << "Stack is empty\n";
        return;
    }
    cout << "Stack elements: ";
for (int i = top; i >= 0; i--)
cout << arr[i] << " ";        cout <<
endl;
}
// Search in stack    void
search(int key) {        for (int i =
top; i >= 0; i--) {        if (arr[i]
== key) {
        cout << key << " found at position " << (top - i) << endl;
return;
    }
}
    cout << key << " not found in stack\n";
}
};

```

```
int main() {  
    Stack s;  
    s.push(10);  
    s.push(20);  
    s.push(30);  
    s.traverse();  
    s.peek();  
  
    s.search(20);  
    s.search(100);  
    s.pop();  
    s.traverse();    return  
0;  
}
```

INPUT AND OUTPUT:

FACULTY OF ENGINEERING & TECHNOLOGY
Data Structure and Algorithms Laboratory (03010503PC02)
ENROLLMENT NO:

PRACTICAL-2

Title:

Infix to Postfix Expression Conversion using Stack

Aim :

To write a program in C++ that converts an infix expression into a postfix expression using stack.

Purpose:

The purpose of this practical is to understand how to use a stack to convert mathematical expressions from infix (normal) form to postfix form. This helps in easier evaluation of expressions by the computer.

Problem Statement:

Convert an infix expression like $(A + B) * C$ into postfix form using stack operations and print the final postfix expression.

Source Code :

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
int precedence(char op){
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return -1; }
string infixToPostfix(string expr) {
    stack<char> st;
    string result;
    for (char ch : expr) {
        if (isalnum(ch)) {
```

```
result += ch;
}
else if(ch == '(') {
    st.push(ch);
} else if(ch == ')') {

while (!st.empty() && st.top() != '(') {
    result += st.top();
    st.pop();
}
if (!st.empty()) st.pop(); // remove '('
} else {
while (!st.empty() && precedence(ch) <= precedence(st.top())) {
    result += st.top();
    st.pop();
}
    st.push(ch); }
}

while (!st.empty()) {
    result += st.top();
    st.pop();
}
return result;
}

int main() {
    string expr;
    cout << "Enter infix expression: ";
    cin >> expr;
    string postfix = infixToPostfix(expr);

    cout << "Postfix expression: " << postfix << endl;
    return 0;
```

Student Sign:

}

INPUT AND OUTPUT:

Student Sign:

PRACTICAL-3

Title :

Implementation of Postfix Expression Evaluation using Stack in C++

Aim :

To evaluate a postfix (Reverse Polish Notation) expression using a stack in C++.

Purpose :

The purpose of this practical is to understand how postfix expressions (where operators follow operands) are evaluated using a stack, which is a Last-In-First-Out (LIFO) data structure. This is commonly used in expression evaluation in compilers and calculators.

Problem Statement :

Write a C++ program to evaluate a postfix expression using a stack.

Accept the expression as input.

Push operands into the stack.

Pop two elements for each operator and apply the operation.

Push the result back to the stack.

At the end, the result should be on top of the stack.

Source Code :

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
int evaluatePostfix(string expr) {
    stack<int> st;
    for (char ch : expr) {
        if (isdigit(ch)) {
            st.push(ch - '0'); // convert char to integer
        } else {
            int val2 = st.top(); st.pop();
```

Student Sign:

```
int val1 = st.top(); st.pop();
switch (ch) {
case '+': st.push(val1 + val2); break;
case '-': st.push(val1 - val2); break;
case '*': st.push(val1 * val2); break;
case '/': st.push(val1 / val2); break;
}
}
}
return st.top();
}
int main() {
string expr;
cout << "Enter postfix expression: ";
cin >> expr;

int result = evaluatePostfix(expr);
cout << "Result: " << result << endl;

return 0;
}
```

INPUT AND OUTPUT:

Student Sign:

PRACTICAL-4

Title :

Implementation of Tower of Hanoi using Stack in C++

Aim :

To simulate the Tower of Hanoi problem using stack data structure in C++ and perform the disk transfer iteratively.

Purpose :

The purpose of this practical is to understand how the Tower of Hanoi problem (traditionally solved recursively) can be simulated using stacks and loops. It demonstrates problem-solving using iterative logic, stack manipulation, and disk transfer rules.

Problem Statement:

Write a C++ program to solve the Tower of Hanoi problem using three stacks representing rods (Source, Helper, Destination).

Accept number of disks as input.

Perform legal moves between rods according to the Tower of Hanoi rules.

Print each move performed.

Source Code :

```
#include <iostream>
#include <stack>
#include <cmath>
using namespace std;

// Function to move a disk between two rods
void moveDisk(stack<int>& from, stack<int>& to, char fromRod, char toRod) {
    int disk;
    if (from.empty()) {
```

Student Sign:

```
disk = to.top(); to.pop();
from.push(disk);

cout << "Move disk " << disk << " from " << toRod << " to " << fromRod << endl;
} else if (to.empty()) {
disk = from.top(); from.pop();
to.push(disk);
cout << "Move disk " << disk << " from " << fromRod << " to " << toRod << endl;
} else if (from.top() > to.top()) {
disk = to.top(); to.pop();
from.push(disk);
cout << "Move disk " << disk << " from " << toRod << " to " << fromRod << endl;
} else {
disk = from.top(); from.pop();
to.push(disk);
cout << "Move disk " << disk << " from " << fromRod << " to " << toRod << endl;
}
}

int main() {
int n;
cout << "Enter number of disks: ";
cin >> n;
stack<int> source, help, destination;

// Push disks to source rod (largest at bottom)
for (int i = n; i >= 1; i--) {
source.push(i);
}

int totalMoves = pow(2, n) - 1;
```

Student Sign:

```
char S = 'S', H = 'H', D = 'D';  
// Swap destination and helper for even number of disks  
if (n % 2 == 0) {  
    swap(H, D);  
}  
// Perform the moves iteratively  
for (int i = 1; i <= totalMoves; i++) {  
    if (i % 3 == 1)  
        moveDisk(source, destination, S, D);  
    else if (i % 3 == 2)  
        moveDisk(source, help, S, H);  
    else  
        moveDisk(help, destination, H, D);  
}  
return 0;  
}
```

INPUT AND OUTPUT:

Student Sign:

PRACTICAL-5

Title: Implementation of Queue and its operations (Enqueue, Dequeue, Traverse, Search)
using Array in C++

Purpose:

The purpose of this practical is to demonstrate the working of a Queue (FIFO – First In First Out) data structure, where elements are added at the rear and removed from the front. Queues are widely used in scheduling, buffering, and resource management systems.

Problem Statement:

Write a C++ program to implement a queue using a linear array.

Perform the following operations:

Enqueue (Insert an element)

Dequeue (Delete an element)

Traverse (Display all elements)

Search (Find an element in the queue)

Source Code:

```
#include <iostream>
using namespace std;
class Queue {
    int* arr;
    int front, rear, capacity;

public:
    Queue(int size) {
        capacity = size;
```

Student Sign:

```
arr = new int[capacity];
front = rear = -1;
}
void enqueue(int value) {
if (rear == capacity - 1) {
cout << "Queue Overflow" << endl;
return;
}
if (front == -1) front = 0;
arr[++rear] = value;
cout << "Enqueued: " << value << endl;
}
void dequeue() {
if (front == -1 || front > rear) {
cout << "Queue Underflow" << endl;
return;
}
cout << "Dequeued: " << arr[front++] << endl;
}

void traverse() {
if (front == -1 || front > rear) {
cout << "Queue is empty" << endl;
return;
}
cout << "Queue elements: ";
for (int i = front; i <= rear; i++)
cout << arr[i] << " ";
cout << endl;
}
```

Student Sign:

```
void search(int value) {  
    for (int i = front; i <= rear; i++) {  
        if (arr[i] == value) {  
            cout << "Found at position: " << (i - front) << endl;  
            return;  
        }  
    }  
    cout << "Element not found" << endl;  
}
```

```
~Queue() {  
    delete[] arr;  
}  
};
```

```
int main() {  
    int size;  
    cout << "Enter queue size: ";  
    cin >> size;
```

```
    Queue q(size);  
    q.enqueue(10);  
    q.enqueue(20);  
    q.enqueue(30);  
    q.traverse();  
    q.search(20);  
    q.dequeue();  
    q.traverse();  
    return 0;
```

Student Sign:



FACULTY OF ENGINEERING & TECHNOLOGY Data
Structure and Algorithms Laboratory (03010503PC02)
ENROLLMENT NO:

INPUT AND OUTPUT:

Student Sign:

PRACTICAL-6

Title : Implement Single Linked lists and its operations (creation insertion deletion traversal search reverse)

Aim:

To write a C++ program that performs operations on a singly linked list, such as creation, insertion, deletion, traversal, searching, and reversing the list.

Purpose:

The purpose of this practical is to understand how data is stored dynamically using linked lists, and how we can perform various operations such as adding, deleting, searching, and reversing nodes in a singly linked list.

Problem Statement :

Write a C++ program to implement a singly linked list and perform the following operations:

Insertion at end

Deletion by value

Traversal (displaying list)

Searching a node

Reversing the list

Source Code :

```
#include <iostream>
using namespace std;
```

```
// Node structure
struct Node {
    int data;
    Node* next;
};
// LinkedList class
```

Student Sign:

Node* head;

public:

LinkedList() {

head = nullptr;

}

// Insert at end

void insert(int val) {

Node* newNode = new Node();

newNode->data = val;

newNode->next = nullptr;

if (head == nullptr) {

head = newNode;

} else {

Node* temp = head;

while (temp->next != nullptr)

temp = temp->next;

temp->next = newNode;

}

}

// Delete a node by value

void deleteNode(int val) {

if (head == nullptr) {

cout << "List is empty\n";

return;

}

// If head needs to be deleted

Student Sign:

```
if(head->data == val) {
Node* temp = head;

head = head->next;
delete temp;
cout << val << " deleted from list\n";
return;
}
Node* current = head;
Node* prev = nullptr;

while (current != nullptr && current->data != val) {
prev = current;
current = current->next;
}
if (current == nullptr) {
cout << val << " not found in list\n";
return;
}
prev->next = current->next;
delete current;
cout << val << " deleted from list\n";
}

// Search for a value
void search(int val) {
Node* temp = head;
int pos = 0;

while (temp != nullptr) {
if (temp->data == val) {
```

Student Sign:

```
cout << val << " found at position " << pos << endl;
```

```
return;
```

```
}
```

```
temp = temp->next;
```

```
pos++;
```

```
}
```

```
cout << val << " not found in list\n";
```

```
}
```

```
// Traverse and display the list
```

```
void traverse() {
```

```
Node* temp = head;
```

```
cout << "Linked List: ";
```

```
while (temp != nullptr) {
```

```
cout << temp->data << " -> ";
```

```
temp = temp->next;
```

```
}
```

```
cout << "NULL\n";
```

```
}
```

```
// Reverse the linked list
```

```
void reverse() {
```

```
Node* prev = nullptr;
```

```
Node* curr = head;
```

```
Node* next = nullptr;
```

```
while (curr != nullptr) {
```

```
next = curr->next;
```

```
curr->next = prev;
```

```
prev = curr;
```

```
curr = next;
```

Student Sign:

```
}  
head = prev;  
cout << "List reversed\n";  
}  
};  
  
// Main function  
int main() {  
    LinkedList list;  
    // Creation and insertion  
    list.insert(10);  
    list.insert(20);  
    list.insert(30);  
    list.insert(40);  
  
    list.traverse(); // 10 -> 20 -> 30 -> 40  
  
    // Search operation  
    list.search(30); // Found  
    list.search(100); // Not Found  
  
    // Deletion operation  
    list.deleteNode(20);  
    list.traverse(); // 10 -> 30 -> 40  
  
    // Reverse operation  
    list.reverse();  
    list.traverse(); // 40 -> 30 -> 10  
  
    return 0;  
}
```

Student Sign:

INPUT AND OUTPUT :

Student Sign:

PRACTICAL-7

Title:

Implementation of Doubly Linked List and its Operations (Creation, Insertion, Deletion, Traversal, Search, Reverse)

Aim:

To write a C++ program to implement a Doubly Linked List and perform basic operations like creation, insertion, deletion, traversal, search, and reverse.

Purpose:

The purpose of this practical is to understand how a doubly linked list works and how data can be stored and accessed in both forward and backward directions. By doing this program, I will learn how to perform different operations like adding, removing, and finding elements in a list, and how to reverse it using pointers.

Problem Statement:

Write a C++ program to create a Doubly Linked List and implement the following operations:

1. Creation of the list
2. Insertion of a new node
3. Deletion of a node
4. Traversal (forward and backward)
5. Search for an element
6. Reverse the list

Source Code :

```
#include <iostream>
using namespace std;

struct Node {
int data;
```

Student Sign:

```
Node* next;  
};
```

```
Node* head = NULL;
```

```
// Function to create a new node
```

```
Node* createNode(int value) {  
Node* newNode = new Node();  
newNode->data = value;  
newNode->prev = NULL;  
newNode->next = NULL;  
return newNode;  
}
```

```
// Insert at end
```

```
void insertEnd(int value) {  
Node* newNode = createNode(value);  
if (head == NULL) {  
head = newNode;  
return;  
}
```

```
Node* temp = head;  
while (temp->next != NULL) {  
temp = temp->next;  
}  
temp->next = newNode;  
newNode->prev = temp;  
}
```

```
// Insert at beginning
```

Student Sign:

```
void insertBegin(int value) {
Node* newNode = createNode(value);
if (head == NULL) {
head = newNode;
return;

}
newNode->next = head;
head->prev = newNode;
head = newNode;
}

// Delete from beginning
void deleteBegin() {
if (head == NULL) {
cout << "List is empty\n";
return;
}
Node* temp = head;
head = head->next;
if (head != NULL) head->prev = NULL;
delete temp;
}

// Delete from end
void deleteEnd() {
if (head == NULL) {
cout << "List is empty\n";
return;
}
```

Student Sign:

```
while (temp->next != NULL) {  
temp = temp->next;  
}  
if (temp->prev != NULL) temp->prev->next = NULL;  
else head = NULL;  
delete temp;  
}
```

```
// Search for a value  
void search(int key) {  
Node* temp = head;  
int pos = 1;  
while (temp != NULL) {  
if (temp->data == key) {  
cout << "Element found at position " << pos << "\n";  
return;  
}  
temp = temp->next;  
pos++;  
}  
cout << "Element not found\n";  
}
```

```
// Display list  
void display() {  
Node* temp = head;  
if (temp == NULL) {  
cout << "List is empty\n";  
return;  
}
```

Student Sign:

```
cout << "List: ";
while (temp != NULL) {
    cout << temp->data << " ";
    temp = temp->next;
}
cout << "\n";
}

// Reverse the list
void reverseList() {
    Node* current = head;
    Node* temp = NULL;
    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if (temp != NULL) head = temp->prev;
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertBegin(5);
    display();
    search(20);
    deleteBegin();
    display();
    deleteEnd();
}
```

Student Sign:

```
display();  
insertEnd(30);  
insertEnd(40);  
display();  
reverseList();  
display();  
return 0;  
}
```

INPUT AND OUTPUT:

Student Sign:

PRACTICAL-8

Title:

Implementation of Binary Search and Interpolation Search in C++

Aim:

To implement and compare Binary Search and Interpolation Search algorithms using C++.

Purpose :

- To understand searching techniques.
- To analyze the working and efficiency of Binary Search and Interpolation Search.
- To learn how these algorithms locate a key in a sorted array.

Problem Statement :

Write a C++ program to search for a key in a sorted array using Binary Search and Interpolation Search. The program should take input from the user and display whether the key is found or not found, along with its position if found.

Source Code :

```
#include <iostream>
using namespace std;
// Binary Search Implementation
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;

        if (arr[mid] < key)
            low = mid + 1;
        else
```

Student Sign:

```
high = mid - 1;
}
return -1;
}

// Interpolation Search Implementation
int interpolationSearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high && key >= arr[low] && key <= arr[high]) {
        if (low == high) {
            if (arr[low] == key)
                return low;
            return -1;
        }

        int pos = low + (((key - arr[low]) * (high - low)) / (arr[high] - arr[low]));

        if (arr[pos] == key)
            return pos;

        if (arr[pos] < key)
            low = pos + 1;
        else
            high = pos - 1;
    }
    return -1;
}

int main() {
    int n, key;
```

Student Sign:

```
cout << "Enter size of array: ";
cin >> n;

int arr[n];
cout << "Enter " << n << " sorted elements:\n";
for (int i = 0; i < n; i++)
    cin >> arr[i];

cout << "Enter element to search (key): ";
cin >> key;

// Binary Search
int bResult = binarySearch(arr, n, key);
if (bResult != -1)
    cout << "Binary Search: Key found at index " << bResult << endl;
else
    cout << "Binary Search: Key not found\n";

// Interpolation Search
int iResult = interpolationSearch(arr, n, key);
if (iResult != -1)
    cout << "Interpolation Search: Key found at index " << iResult << endl;
else
    cout << "Interpolation Search: Key not found\n";

return 0;
}
```

Student Sign:

Input and Output :

Student Sign:

PACTICAL-9

Title:

Implementation of Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, and Merge Sort in C++

Aim:

To implement and compare different sorting algorithms in C++.

Purpose:

- To understand various sorting techniques.
- To learn how each algorithm works and analyze their time and space complexities.
- To compare the performance of different sorting algorithms on the same dataset.

Problem Statement :

Write a C++ program to implement the following sorting algorithms:

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Merge Sort

The program should:

- Take an array as input from the user.
- Sort the array using each sorting algorithm.
- Display the sorted array for each algorithm.

Source Code :

```
#include <iostream>
using namespace std;

// Bubble Sort
void bubbleSort(int arr[], int n) {
```

Student Sign:

```
for (int i = 0; i < n - 1; i++)  
    for (int j = 0; j < n - i - 1; j++)  
        if (arr[j] > arr[j + 1])
```

```
        swap(arr[j], arr[j + 1]);
```

```
    }
```

```
// Insertion Sort
```

```
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        int key = arr[i], j = i - 1;  
        while (j >= 0 && arr[j] > key)  
            arr[j + 1] = arr[j--];  
        arr[j + 1] = key;  
    }  
}
```

```
// Selection Sort
```

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++)  
            if (arr[j] < arr[minIndex])  
                minIndex = j;  
        swap(arr[minIndex], arr[i]);  
    }  
}
```

```
// Quick Sort
```

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high], i = low - 1;  
    for (int j = low; j < high; j++)
```

Student Sign:

```
if (arr[j] < pivot) swap(arr[++i], arr[j]);  
swap(arr[i + 1], arr[high]);  
return i + 1;
```

```
}
```

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

// Merge Sort

```
void merge(int arr[], int l, int m, int r) {  
    int n1 = m - l + 1, n2 = r - m;  
    int L[n1], R[n2];  
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];  
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];  
    int i = 0, j = 0, k = l;  
    while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];  
    while (i < n1) arr[k++] = L[i++];  
    while (j < n2) arr[k++] = R[j++];  
}
```

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
        merge(arr, l, m, r);  
    }
```

Student Sign:

```
}  
  
// Print Array  
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++) cout << arr[i] << " ";  
    cout << endl;  
}  
  
int main() {  
    int n;  
    cout << "Enter size of array: ";  
    cin >> n;  
    int arr[n], temp[n];  
    cout << "Enter " << n << " elements:\n";  
    for (int i = 0; i < n; i++) cin >> arr[i];  
  
    // Bubble Sort  
    copy(arr, arr + n, temp);  
    bubbleSort(temp, n);  
    cout << "Bubble Sort: "; printArray(temp, n);  
  
    // Insertion Sort  
    copy(arr, arr + n, temp);  
    insertionSort(temp, n);  
    cout << "Insertion Sort: "; printArray(temp, n);  
  
    // Selection Sort  
    copy(arr, arr + n, temp);  
    selectionSort(temp, n);  
    cout << "Selection Sort: "; printArray(temp, n);
```

Student Sign:

```
// Quick Sort
copy(arr, arr + n, temp);
quickSort(temp, 0, n - 1);
cout << "Quick Sort: "; printArray(temp, n);
```

```
// Merge Sort
copy(arr, arr + n, temp);
mergeSort(temp, 0, n - 1);
cout << "Merge Sort: "; printArray(temp, n);
return 0;
}
```

Input:

Enter size of array: 6

Enter 6 elements: 50 20 10 40 30 60

Input and Output :

Student Sign:

PRACTICAL-10

Title :

Implementation of Binary Search Tree (BST) and its Operations in C++

Aim :

To implement a Binary Search Tree (BST) in C++ and perform basic operations such as creation, insertion, and deletion of nodes.

Purpose :

The purpose of this experiment is to understand how Binary Search Trees work and to demonstrate

their functionality by implementing common operations. BSTs are widely used in searching and sorting applications because they provide efficient data storage and retrieval.

Problem Statement :

Design and implement a program in C++ to:

1. Create a Binary Search Tree (BST).
2. Insert nodes into the BST while maintaining its properties.
3. Delete a node from the BST.
4. Display the tree using Inorder traversal to verify correctness.

Source Code :

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;
    Node* left;
    Node* right;
};
// Function to create a new node
```

Student Sign:

```
Node* createNode(int value) {  
Node* newNode = new Node();  
newNode->data = value;  
newNode->left = newNode->right = nullptr;  
return newNode;  
}
```

// Insert function

```
Node* insert(Node* root, int value) {  
if (root == nullptr) {  
return createNode(value);  
}  
if (value < root->data) {  
root->left = insert(root->left, value);  
} else if (value > root->data) {  
root->right = insert(root->right, value);  
}  
return root;  
}
```

// Find the minimum node

```
Node* findMin(Node* root) {  
while (root && root->left != nullptr) {  
root = root->left;  
}  
return root;  
}
```

// Delete function

```
Node* deleteNode(Node* root, int value) {  
if (root == nullptr) return root;  
if (value < root->data) {
```

Student Sign:



```
root->left = deleteNode(root->left, value);
} else if (value > root->data) {
root->right = deleteNode(root->right, value);
} else {
// Node with only one child or no child
if (root->left == nullptr) {
Node* temp = root->right;
delete root;
return temp;

} else if (root->right == nullptr) {
Node* temp = root->left;
delete root;
return temp;
}
// Node with two children
Node* temp = findMin(root->right);
root->data = temp->data;
root->right = deleteNode(root->right, temp->data);
}
return root;
}

// Inorder Traversal
void inorder(Node* root) {
if (root != nullptr) {
inorder(root->left);
cout << root->data << " ";
inorder(root->right);
}
}

// Main function
```

Student Sign:

```
int main() {  
    Node* root = nullptr;  
    // Insert elements  
    root = insert(root, 50);  
    root = insert(root, 30);  
    root = insert(root, 70);  
    root = insert(root, 20);  
    root = insert(root, 40);  
    root = insert(root, 60);  
    root = insert(root, 80);  
  
    cout << "Inorder traversal of the BST: ";  
    inorder(root);  
    cout << endl;  
    cout << "Deleting 20\n";  
    root = deleteNode(root, 20);  
    inorder(root);  
    cout << endl;  
  
    cout << "Deleting 30\n";  
    root = deleteNode(root, 30);  
    inorder(root);  
    cout << endl;  
  
    cout << "Deleting 50\n";  
    root = deleteNode(root, 50);  
    inorder(root);  
    cout << endl;  
  
    return 0;  
}
```

Student Sign:



INPUT AND OUTPUT :

Student Sign:

PRACTICAL-11

Title :

Implementation of Tree Traversals (Preorder, Inorder, Postorder) on a Binary Search Tree in C++

Aim :

To implement and demonstrate different tree traversal techniques (Preorder, Inorder, Postorder) on a Binary

Search Tree (BST) in C++.

Purpose :

The purpose of this experiment is to understand the different ways of traversing a Binary Search Tree. Each

traversal provides a unique way to process or display the elements of the tree:

- Inorder: Gives elements in sorted order (for BST).
- Preorder: Useful for copying the tree or prefix expressions.
- Postorder: Useful for deleting the tree or postfix expressions.

Problem Statement :

Write a C++ program to:

1. Create a Binary Search Tree.
2. Insert nodes while maintaining BST properties.
3. Perform and display traversals: Preorder, Inorder, Postorder.

Source Code (C++) :

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
int data;
```

Student Sign:



```
Node* left;
Node* right;
};
// Create a new node
Node* createNode(int value) {
Node* newNode = new Node();
newNode->data = value;

newNode->left = newNode->right = nullptr;
return newNode;
}
// Insert into BST
Node* insert(Node* root, int value) {
if (root == nullptr) {
return createNode(value);
}
if (value < root->data) {
root->left = insert(root->left, value);
} else if (value > root->data) {
root->right = insert(root->right, value);
}
return root;
}
// Inorder Traversal (Left → Root → Right)
void inorder(Node* root) {
if (root != nullptr) {
inorder(root->left);
cout << root->data << " ";
inorder(root->right);
}
}
```

Student Sign:

```
// Preorder Traversal (Root → Left → Right)
void preorder(Node* root) {
    if (root != nullptr) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder Traversal (Left → Right → Root)
void postorder(Node* root) {
    if (root != nullptr) {
        postorder(root->left);

        postorder(root->right);
        cout << root->data << " ";
    }
}

// Main function
int main() {
    Node* root = nullptr;
    // Insert elements into BST
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);
    cout << "Inorder Traversal: ";
    inorder(root);
}
```

Student Sign:



```
cout << endl;
cout << "Preorder Traversal: ";
preorder(root);
cout << endl;
cout << "Postorder Traversal: ";
postorder(root);
cout << endl;
return 0;
}
```

INPUT AND OUTPUT :

Student Sign:

PRACTICAL-12

Title :

Graph Representation using Adjacency List and Adjacency Matrix with Traversals (BFS & DFS) in C++

Aim :

To implement and represent graphs using Adjacency List and Adjacency Matrix and perform basic graph

traversals (BFS and DFS) in C++.

Purpose :

Graphs are a fundamental data structure used to represent relationships between objects.

- Adjacency Matrix provides a simple representation but may use more memory.
- Adjacency List is space-efficient for sparse graphs.
- BFS (Breadth-First Search) is useful for finding the shortest path in unweighted graphs.
- DFS (Depth-First Search) is useful for exploring paths deeply and solving connectivity problems.

Problem Statement :

Write a C++ program to:

1. Represent a graph using Adjacency List and Adjacency Matrix.
2. Perform Breadth-First Search (BFS) and Depth-First Search (DFS) traversals.

Source Code (C++):

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
class Graph {
```

Student Sign:



int V;

// Number of vertices vector<vector<int>>

adjList; // Adjacency List

vector<vector<int>> adjMatrix; // Adjacency Matrix

public:

Graph(int vertices) {

V = vertices;

adjList.resize(V);

adjMatrix.resize(V, vector<int>(V, 0));

}

// Add Edge (Undirected Graph)

void addEdge(int u, int v) {

// For adjacency list

adjList[u].push_back(v);

adjList[v].push_back(u);

// For adjacency matrix

adjMatrix[u][v] = 1;

adjMatrix[v][u] = 1;

}

// Print adjacency list

void printAdjList() {

cout << "Adjacency List:" << endl;

for (int i = 0; i < V; i++) {

cout << i << " -> ";

for (int v : adjList[i]) {

cout << v << " ";

}

cout << endl;

}

}

// Print adjacency matrix

void printAdjMatrix() {

cout << "Adjacency Matrix:" << endl;

for (int i = 0; i < V; i++) {

for (int j = 0; j < V; j++) {

cout << adjMatrix[i][j] << " ";

}

cout << endl;

}

}

// BFS Traversal

Student Sign:

```
void BFS(int start) {
vector<bool> visited(V, false);
queue<int> q;

visited[start] = true;
q.push(start);
cout << "BFS Traversal starting from " << start << ": ";
while (!q.empty()) {
int node = q.front();
q.pop();
cout << node << " ";

for (int neighbor : adjList[node]) {
if (!visited[neighbor]) {
visited[neighbor] = true;
q.push(neighbor);
}
}
}
cout << endl;
}

// DFS Helper
void DFSUtil(int node, vector<bool>& visited) {
visited[node] = true;
cout << node << " ";

for (int neighbor : adjList[node]) {
if (!visited[neighbor]) {
DFSUtil(neighbor, visited);
}
}
}

// DFS Traversal
void DFS(int start) {
vector<bool> visited(V, false);
cout << "DFS Traversal starting from " << start << ": ";
DFSUtil(start, visited);
cout << endl;
}
```

Student Sign:



```
};  
// Main function  
int main() {  
int V = 5; // Number of vertices  
Graph g(V);  
// Add edges  
g.addEdge(0, 1);  
g.addEdge(0, 2);  
g.addEdge(1, 3);  
g.addEdge(1, 4);  
g.addEdge(2, 4);  
// Representations  
g.printAdjList();  
cout << endl;  
g.printAdjMatrix();  
cout << endl;  
// Traversals  
g.BFS(0);  
g.DFS(0);  
return 0;  
}
```

INPUT AND OUTPUT :

Student Sign: